Release 2026-11-09: Windows laptop and VM

What to do on the Windows laptop (Command Prompt) and the Ubuntu VM (bash) to bring them to today's code.

On the VM: **Postgres stays the database** (`DB_BACKEND=postgres`). The Cosmos key there is read-only, and every mode that uses Cosmos writes to it. **Chart runs go through Service Bus** (`RUN_MODE=queue`): the API queues each chart and the `worker` containers run them.

Order: download → push → VM pull → back up → settings → database → restart → check → re-run charts → utilities.

---

1. Windows laptop: download from GitHub

Two repos: the main one and the client UI.

1. Open https://github.com/abhinavdg-ever/model-repo → **Code** → **Download ZIP** (saves `model-repo-main.zip` in Downloads).
2. Open https://github.com/abhinavdg-ever/ui-temp → **Code** → **Download ZIP** (saves `ui-temp-main.zip`).

```
:: Windows (Command Prompt)
cd /d %USERPROFILE%\Downloads
rmdir /s /q model-repo-main 2>nul
tar -xf model-repo-main.zip
rmdir /s /q ui-temp-main 2>nul
tar -xf ui-temp-main.zip
```

2. Windows laptop: into the Azure DevOps folder and push

`prepare_for_devops.ps1` copies the code in, leaving out training / scripts / tests and keeping your `.env` files, model weights and logs. It runs from Command Prompt like this:

```
:: Windows (Command Prompt)
powershell -NoProfile -ExecutionPolicy Bypass -File "%USERPROFILE%\Downloads\model-repo-main\deploy\prepare_for_devops.ps1" -Source "%USERPROFILE%\Downloads\model-repo-main" -Target "C:\work\advantmed-devops"
```

Then push `C:\work\advantmed-devops` to Azure DevOps as usual. Deliver `ui-temp-main` (the client UI) the way you usually do.

No new model weights this release: nothing to copy into `core-pipeline/models/`.

3. VM: pull

```
# Ubuntu VM
cd ~/advantmed-imaging-pipeline
git pull                                    # from Azure DevOps
```

4. VM: back up the database (Postgres in Docker)

The ground-truth copy in §9 replaces rows, and the re-run in §8 rewrites verdicts, so take a backup first. Set the three names once per terminal: the Postgres container, its user, and the database (`imaging_outputs` unless yours is named differently).

```
# Ubuntu VM
docker ps --format '{{.Names}}  {{.Image}}' | grep -i postgres     # find the container name
PG=<container name>          # e.g. postgres
PGU=<db user>                # the user in DATABASE_URL
PGDB=imaging_outputs         # the database in DATABASE_URL

docker exec $PG pg_dump -U $PGU -d $PGDB -Fc > ~/backup_$(date +%Y%m%d_%H%M).dump
ls -lh ~/backup_*.dump | tail -1                                    # should not be tiny
```

```
# the client's database too (the ground truth comes from it)
docker exec $PG pg_dump -U $PGU -d imaging_outputs_client -Fc > ~/backup_client_$(date +%Y%m%d_%H%M).dump
```

To put a backup back (only if something went wrong):

```
docker exec -i $PG pg_restore -U $PGU -d $PGDB --clean --if-exists < ~/backup_<stamp>.dump
```

5. VM: settings

The `.env` files are not in git. Add any line below that a file does not have yet, in the section named; keep each `.env` in the same order as its `.env.example`.

`core-pipeline/.env`

```
# --- 14. PLATFORM ---
DB_BACKEND=postgres
COSMOS_ENDPOINT=
COSMOS_KEY=
COSMOS_DATABASE=imaging
SERVICEBUS_CONNECTION_STRING=<the Service Bus connection string>
SERVICEBUS_QUEUE=chart-runs
ORCHESTRATOR_API_KEY=
API_KEY_REQUIRED=false
RUN_MODE=queue
WORKER_LOCK_RENEW_HOURS=6

# --- 15. RESOURCES ---
RESOURCES_FROM_BLOB=false
RESOURCES_CONTAINER=imaging-models
RESOURCES_MODELS=
RESOURCES_CANON_FROM_BLOB=false
```

Remove `METADATA_ROOT=` if it is still there (no longer read). The queue `chart-runs` must have **sessions enabled** (it was created that way).

`review-ui/.env` (and the client UI's `.env`, the same lines)

```
# --- 6. PLAYGROUND ---  (after PLAYGROUND_CONTAINER)
ORCHESTRATOR_API_KEY=
# --- 8. PLATFORM ---
DB_BACKEND=postgres
COSMOS_ENDPOINT=
COSMOS_KEY=
COSMOS_DATABASE=imaging
```

`db-utilities/.env` (new folder: make it once)

```
cd ~/advantmed-imaging-pipeline/db-utilities
cp .env.example .env
# DATABASE_URL and AZURE_STORAGE_* as in core-pipeline/.env; DB_BACKEND=postgres
```

Check that the keys match the examples (prints key names only, never values):

```
cd ~/advantmed-imaging-pipeline
for d in core-pipeline review-ui db-utilities; do
  diff <(grep -o '^[A-Z_]*=' $d/.env.example) <(grep -o '^[A-Z_]*=' $d/.env) >/dev/null \
    && echo "$d: same keys" || { echo "$d: differs"; diff <(grep -o '^[A-Z_]*=' $d/.env.example) <(grep -o '^[A-Z_]*=' $d/.env); }
done
```

6. VM: database upgrade

`migrate.sql` pulls in the files under `schema/migrations/`, so the folder is copied into the container first. Safe to run every release.

```
        # Ubuntu VM
        cd ~/advantmed-imaging-pipeline
        docker cp schema $PG:/tmp/schema
        docker exec $PG psql -U $PGU -d $PGDB -v ON_ERROR_STOP=1 -f /tmp/schema/migrate.sql
        docker exec $PG psql -U $PGU -d $PGDB -c "SELECT count(*) AS stages FROM pipeline_stage;"   # 15
```

7. VM: restart and check

```
        # Ubuntu VM
        # core-pipeline: new Python packages (azure-cosmos, azure-servicebus), so --build.
        # Each worker runs one chart at a time with its own models (several GB): start with 2.
        cd ~/advantmed-imaging-pipeline/core-pipeline
        docker compose up -d --build --scale worker=2
        curl -fsS localhost:8001/ready && echo pipeline ready
        curl -s localhost:8001/health | python3 -c "import json,sys; h=json.load(sys.stdin); print('db',
h.get('db_backend')); print('run_mode', h.get('run_mode')); print('queue', h.get('queue'))"
        # expect: db {'configured': 'postgres', 'active': 'postgres', ...}, run_mode queue, the queue reachable
        docker compose logs --tail 20 worker                                  # "queue worker starting"

        # review-ui
        cd ../review-ui
        docker compose up -d --build
        curl -fsS localhost:4000/docs >/dev/null && echo review-ui up

        # db-utilities: build its image once (it runs only when you load a file)
        cd ../db-utilities
        docker compose build
```

The client UI, in its own folder on the VM: `git pull && docker compose up -d --build`.

8. Re-run charts for the new rules

What changed for results: the blank / junk rules (`docs/specials/BLANK_JUNK_LOGIC.md`), the page-type tie (Progress Note first) and rotation. `skip_ocr: true` re-runs every stage on the OCR already stored, so **no Doc Intel bill**. The rotation stage still runs; only a page whose rotation now comes out different is OCR'd again. With `RUN_MODE=queue` the call returns at once and the workers take the charts one by one.

```
        # Ubuntu VM (input_path = the Blob folder the charts came from)
        curl -X POST localhost:8001/api/charts/batch-run -H 'Content-Type: application/json' \
          -d '{"input_type":"blob","container_name":"imaging-pipeline","input_path":"raw_input/Batch1/Run1",
               "output_path":"Processed/Batch1/Run1","skip_ocr":true}'

        :: Windows (Command Prompt), against the VM
        curl -X POST http://172.20.4.170:8001/api/charts/batch-run -H "Content-Type: application/json" -d "
{\"input_type\":\"blob\",\"container_name\":\"imaging-
pipeline\",\"input_path\":\"raw_input/Batch1/Run1\",\"output_path\":\"Processed/Batch1/Run1\",\"skip_ocr\":true}"
```

Add `"chart_list": ["<chart>", ...]` to re-run only some charts. Progress: the call answers with a run id per chart; `curl -s localhost:8001/api/runs/<run id>` (queued / running / done), `curl -s localhost:8001/api/charts/by-name/<chart>`, and `docker compose logs -f worker`.

9. The two utilities (VM)

Both run on the VM host from the core-pipeline venv. If the VM has no venv yet, make it once:

```
cd ~/advantmed-imaging-pipeline/core-pipeline
python3.12 -m venv .venv && source .venv/bin/activate && pip install -r requirements-basic.txt
```

They read `DATABASE_URL` from `core-pipeline/.env`. That URL is written for the containers; if its host is a Docker name (e.g. `@postgres:5432`), give the host's address for these runs: `export DATABASE_URL='postgresql://USER:PASSWORD@localhost:5432/imaging_outputs'`.

Mark charts "Special" (`utilities/tag_chart.py`; History and Accuracy filter by tag)

```
cd ~/advantmed-imaging-pipeline/core-pipeline && source .venv/bin/activate
python ../utilities/tag_chart.py 52743839_44976074 --tag Special
python ../utilities/tag_chart.py --from-file charts.txt --tag Special     # one chart per line
python ../utilities/tag_chart.py --list-tags
```

The client's ground truth into ours (`utilities/pull_client_ground_truth.py`). The client database is on the same Postgres, so its URL is ours with the database name changed.

```
cd ~/advantmed-imaging-pipeline/core-pipeline && source .venv/bin/activate
export CLIENT_DATABASE_URL='postgresql://USER:PASSWORD@localhost:5432/imaging_outputs_client'
python ../utilities/pull_client_ground_truth.py              # dry run: rows per table, ours vs theirs
python ../utilities/pull_client_ground_truth.py --apply      # backs ours up, then copies
python ../utilities/pull_client_ground_truth.py --list-backups
python ../utilities/pull_client_ground_truth.py --restore gt_backup_<stamp>   # undo
```

Check it in the database:

```
docker exec $PG psql -U $PGU -d $PGDB -c "SELECT count(*) AS pages, count(DISTINCT chart_name) AS charts FROM page_ground_truth;"
```

Then review-ui → Accuracy View → **Refresh**, and **Download Page Level Accuracy** for the page-by-page Match / No Match file.

10. Later: Cosmos (no code release needed)

When the VM gets a read-write Cosmos key: `DB_BACKEND=both` plus `COSMOS_*` in core-pipeline, review-ui and db-utilities, after `utilities/migrate_postgres_to_cosmos.py --all` has copied the history. A week on `both` with matching counts, then `DB_BACKEND=cosmos`. The API key: `API_KEY_REQUIRED=true` and the same `ORCHESTRATOR_API_KEY` in core-pipeline and review-ui.